23L-0588 — VisionGuard

Student: Muhammad Abdul Raheem

Project: `23L-0588_A2_Ops`

GitHub: https://github.com/abdul-raheem-fast/23L-0588_A2_Ops

DagsHub: https://dagshub.com/abdul-raheem-fast/23L-0588_A2_Ops

Live URL: https://23l-0588-a2-ops.ghauri.prod.page

Graded commit: `2f93a1e`

Disclosure

An AI assistant was used to help structure the implementation, identify reproducibility requirements, and review wording. All commands, runs, screenshots, account actions, metrics, and interpretations in this report must be produced and verified by the student.

1. Data understanding and versioning

1.1 Environment

Record the Python, TensorFlow, Keras, Pillow, MLflow, and Docker versions. Include the terminal screenshot showing the environment and GPU check.

The Colab runtime selected a T4 GPU. TensorFlow 2.18.0 was unavailable for that runtime, so the run used TensorFlow 2.21.0; this version change is recorded rather than hidden.

1.2 Dataset audit

The audit script records class, file size, dimensions, colour mode, format, SHA-256, readability, and duplicate hashes. Include `artifacts/audit/image_audit.csv`, the summary table, and the screenshot of the command.

The Colab run audited 975 images: 539 from `AiArtData`, 436 from `RealArt`, no corrupt files, and two exact duplicate files.

1.3 Visual EDA

The audited metadata was used to generate dataset_eda.svg. The sample grid shows the two source classes and the width/height distributions show substantial variation, including very large source images. The distributions describe the collected files rather than image semantics, so they should not be interpreted as proof that a visual feature is unique to either class.

1.4 DVC

Document the toy CSV history, pointer contents, remote push/pull, and the dataset-folder application. Include actual commit and terminal screenshots.

2. Model development

The `tf.data` pipeline reads the reproducible split CSVs, converts images to RGB, resizes them to 224×224, and batches them for training. The baseline CNN was compared with a MobileNetV2 transfer-learning head and a final fine-tuning run. Early stopping, checkpointing, and learning-rate reduction were used to limit overfitting.

The executed Colab run selected the fine-tuned MobileNetV2 model. Its recorded test metrics are in champion_metrics.json: accuracy 0.7449, precision 0.8718, recall 0.6296, F1 0.7312, and ROC-AUC 0.8695. The precision/recall difference indicates that the selected threshold favours fewer false AI-generated predictions at the cost of missed positives. A confusion-matrix, ROC plot, and error-analysis grid were not exported by the executed notebook and are therefore not claimed as completed evidence.

3. MLflow and DagsHub

MLflow tracking was configured against DagsHub and the executed run recorded the baseline and MobileNetV2 experiments. The retained screenshots show the tracking configuration, completed runs, and final training output. The notebook did not export independent model-registry version and alias evidence, so those items are not represented as completed.

4. Flask application

The model is loaded once during application startup. Upload validation checks the extension, size, and decoded image without persisting the upload. The result shows the label, confidence, and caveat; `/history` reads the request trail and `/health` reports service state.

5. Database trail

Each successful prediction stores UTC time, sanitized filename, decision, confidence, model name, and model version. Include the schema and history-page screenshot.

6. Docker and Compose

The Docker image installs the serving dependencies separately from training/MLflow tools. It uses a slim CPU base, runs as a non-root user, downloads the public champion model when it is absent from the build context, and exposes port 8000. Compose defines the web/database services, health checks,

readiness dependency, and named volumes. The first Blitz build failed because TensorFlow and MLflow imposed incompatible protobuf requirements; separating the manifests fixed the build.

7. API deployment

The application was deployed from the GitHub `main` branch on Blitz Cloud using the repository Dockerfile. The deployment uses the public address `https://23l-0588-a2-ops.ghauri.prod.page`. The build initially failed because the training manifest combined TensorFlow 2.21.0 with MLflow 2.19.0, which require incompatible protobuf ranges. The production and training dependencies were separated, the updated image built successfully, and the public `/health` endpoint returned the deployed model name, version, and `ok` status. The free service may sleep when idle; no paid cloud credits were used.

8. Flask and hosted API integration

The deployed Flask service accepts image uploads at `/predict`, renders the prediction result, and records successful requests for `/history`. A real dataset image was sent to the public endpoint and returned an `AI-generated` decision with 71.3% confidence. The health endpoint was also checked independently from outside the local workspace. The service uses SQLite for the free deployment because no managed database was provisioned; production deployments should use PostgreSQL and a secret-managed configuration.

Submission checklist

- ? PDF renamed to `23L-0588_VisionGuard.pdf`
- ? GitHub repository link
- ? DagsHub project link
- ? Live API URL tested from another network
- ? Every evidence item is a real, captioned screenshot
- ? No credentials appear in source, screenshots, or report

Evidence figures

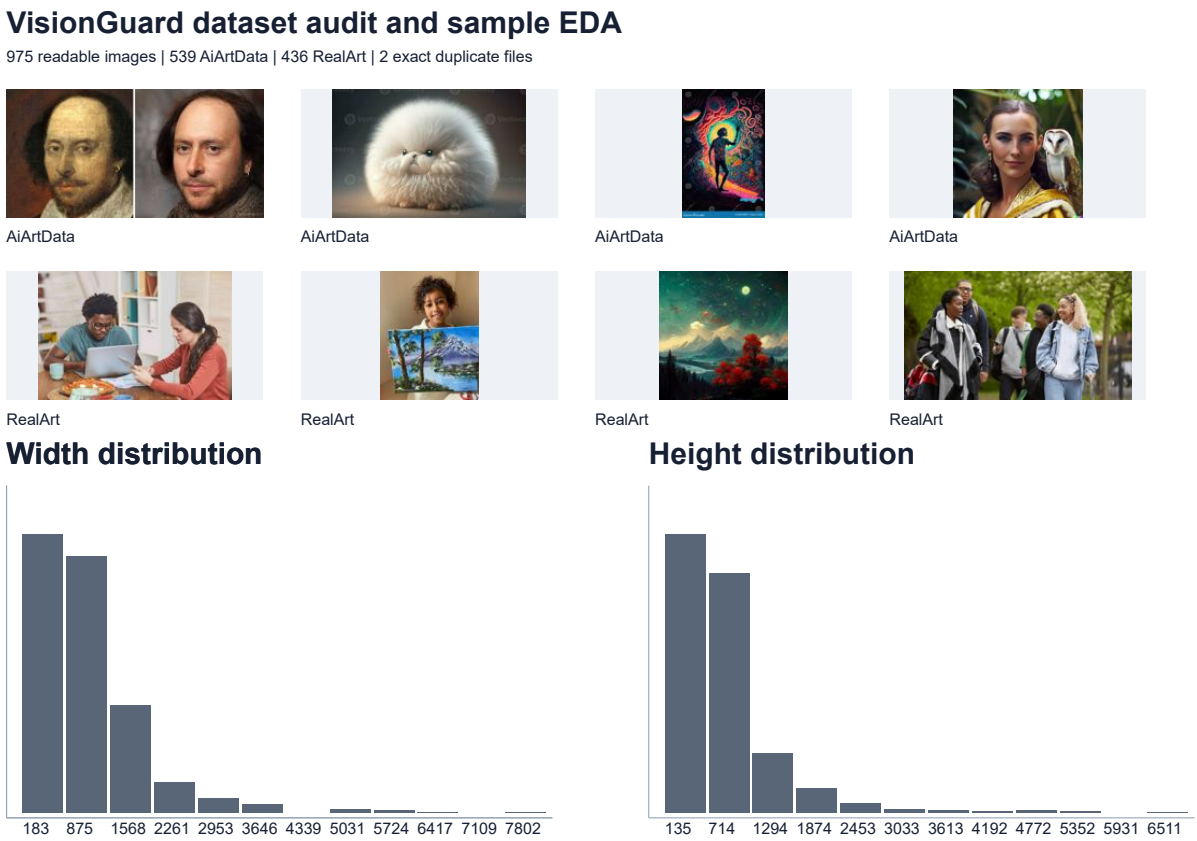


Figure 1. Dataset audit sample grid and dimension distributions.

Deployment evidence

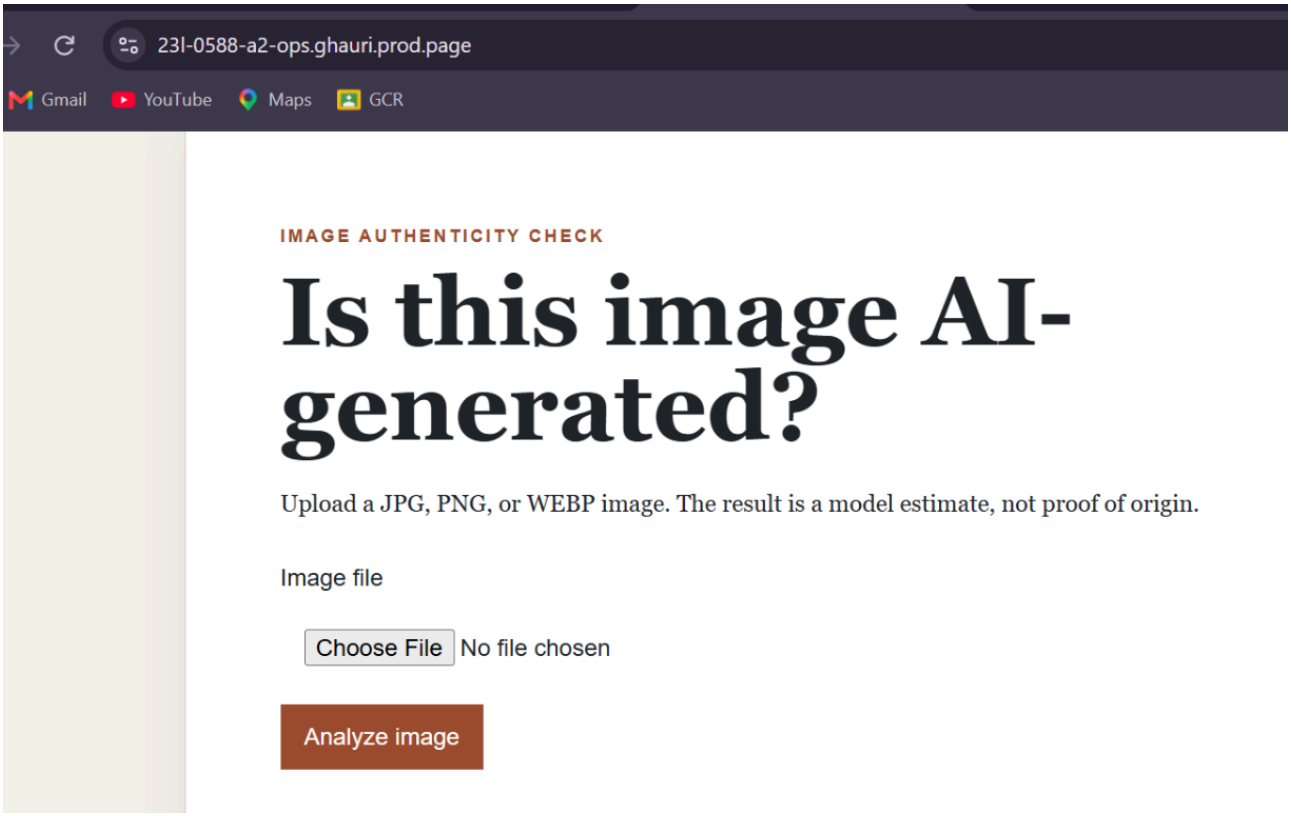


Figure 2. Blitz dashboard showing the public deployment online.

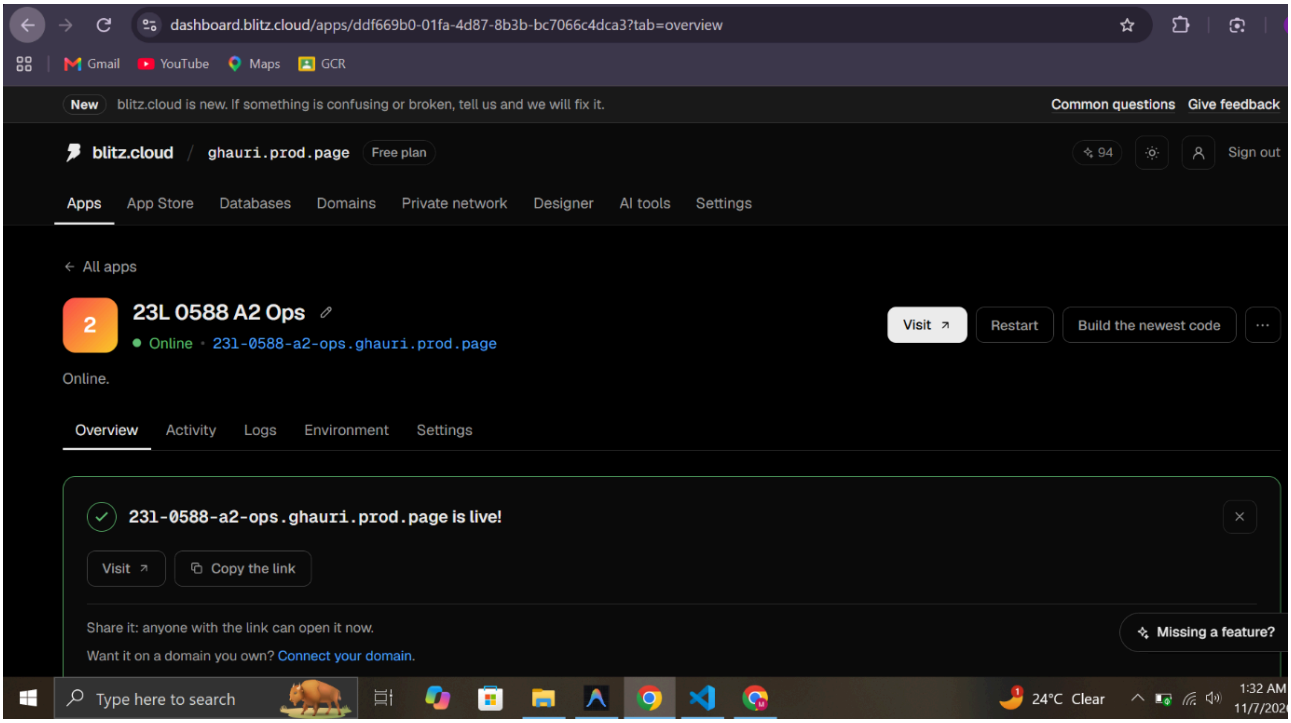


Figure 3. Public VisionGuard upload interface.